

TransitFlow Vector / RAG Design說明

Section 4 — Vector / RAG Design

1. Embedded Data and Cosine Similarity Rationale

在 TransitFlow 系統中，我們將所有營運政策文件（例如：退票規則、票種說明、旅客行為準則等）透過嵌入模型轉換為向量，並儲存於 PostgreSQL 的 `policy_documents` 表格中。

為了進行語意檢索，我們選擇 **Cosine Similarity** 作為距離度量標準。

- **Justification:** 餘弦相似度的數學本質是計算兩個向量在多維空間中的夾角方向，而完全忽略向量的長度。這在 RAG 架構中極為關鍵，因為使用者的提問通常很短（例如："Can I bring a bike?"），而資料庫中的政策文件通常極長。即便兩者字數差距懸殊，只要它們在語意空間上的方向一致，餘弦相似度就能精準判定它們具備高度關聯性，從而完美解決長短文本的語意匹配問題。

2. The Full RAG Pipeline

我們的檢索增強生成流程分為以下四個完整階段：

1. **Query Embedding:** 當使用者提出與政策相關的自然語言問題時，系統的 Agent 會觸發 `search_policy` 工具，並呼叫 Ollama 嵌入模型將該使用者的字串轉換為一組高維度浮點數向量。
2. **Similarity Search:** 將生成的查詢向量送入 PostgreSQL，利用 `pgvector` 擴充套件與 `HNSW` 索引（採用 `vector_cosine_ops`），在 `policy_documents` 表格中快速計算並找出餘弦相似度最高的 Top-K 筆政策向量。
3. **Retrieved Documents:** 資料庫回傳匹配成功的紀錄，系統萃取這些紀錄的原始文本內容、標題與分類。
4. **LLM Prompt & Answer:** 將檢索到的政策文本作為 Context 注入到 LLM 的 Prompt 中。LLM 基於這些真實且具體的營運規則進行推論，最終生成準確、無幻覺的自然語言解答回覆給使用者。

3. Embedding Dimension Choice and Provider Constraints

- **Dimension Choice:** 在本專案中，我們選擇使用 Ollama（預設模型為 `nomie-embed-text`）作為嵌入提供者。因此，我們在 `schema.sql` 中明確將 `embedding` 欄位的資料型態定義為 `vector(768)`，以完美吻合 Ollama 輸出的 768 維度限制。
- **Consequence of Switching Providers:** 若在系統已經 Seeding（播種）完成後，突然將 LLM Provider 切換為 Gemini（其 `gemini-embedding-001` 模型輸出為 3072 維），將會引發嚴重的維度不匹配災難。
 1. 寫入失敗：PostgreSQL 會直接拒絕 3072 維的新向量寫入 `vector(768)` 的欄位，引發 SQL Exception。

2. 檢索崩潰:在數學上,無法對兩個維度不同的向量計算餘弦夾角。即便移除了資料庫的欄位長度限制, 768 維的舊文件也無法與 3072 維的新查詢進行比對, 導致整個 HNSW 索引與檢索系統完全失效。唯一解法是 **DROP** 掉整張資料表並重新呼叫 API 進行全庫 Embedding。